University Semester Project Defense • Computer Science

High-Performance, Modular Console Architecture in Standard C++17 with Persistent File Handling

ISO C++17 Standard

Zero External Dependencies

3-Tier Layered Architecture

MinGW-w64 GCC 16.1.0

Course: Object-Oriented Programming & Data Structures

Project Scope: Student Information, Attendance & Examinations

Demonstration Mode: Live Code, UI & Viva Defense

Academic Year: 2026 – 2027

Executive Summary & Problem Statement

Why traditional educational management fails and how modern C++ solves it

The Administrative Problem

Educational institutions regularly face administrative bottlenecks in tracking students:

- **Data Redundancy:** Duplicate records, conflicting IDs, and untracked duplicate attendance logs.
- **Orphaned History:** Deleting students leaves abandoned exam results and attendance entries across disconnected files.
- **Software Bloat:** Overengineered web/SQL apps require heavy servers, runtimes, and complex setups.
- **Terminal Fragility:** Standard C++ student programs crash when invalid input is typed into `cin`.

The C++17 Solution

A native, self-contained, lightning-fast application engineered from first principles:

- **Strictly Scoped:** Focuses exclusively on Student Records, Attendance, and Examinations without unnecessary bloat.
- **Zero Dependencies:** 100% ISO C++17 Standard Library; no Qt, no SQLite, no third-party runtimes.
- **Referential Integrity:** Real-time cascading deletions purge associated records across all files automatically.
- **Defensive Parsing:** Crash-proof input sanitization using custom stream tokenization.

Key System Metrics

3
Core Modules Managed

3
University Special Features

Strict Scope & Special Features

Clear boundaries, modular subsystems, and three university-grade highlights

The 3 Core Modules (Mandated Scope)

Module	Core Responsibilities	Storage
1. Student Info	Add, Display, Search by ID, Update, Delete with duplicate protection	`data/students.txt`
2. Attendance	Mark Present/Absent, ISO Date check, Duplicate block, Percentage	`data/attendance.txt`
3. Examinations	Enter marks, dynamic percentage, automated grading (A+ to F)	`data/results.txt`

Strictly Excluded Unnecessary Features

To preserve high engineering quality, the following were intentionally excluded:

- ✖ **No Fee Management**
- ✖ **No Library Module**
- ✖ **No Login / Auth Bloat**
- ✖ **No Teacher Portal**

The 3 Extra University Features

1. Student Profile Card (Module 1)

An all-in-one console dashboard integrating student identity, attendance eligibility, and cumulative academic performance in a single character-aligned view.

2. Academic Report Card (Module 3)

Official transcript with subject-wise marks, individual grades, total obtained/maximum score, aggregate percentage, and PASS/FAIL evaluation.

3. Class Merit List (Module 3)

Automated ranked leaderboard with cohort filtering. Implements standard competition ranking where identical percentages share the same rank.

Technology Stack & Compiler Toolchain

Strict compliance with ISO C++17 standards, GCC compilation, and Win32 virtual terminal

Core Language & Toolchain

- **Language:** Standard C++ (ISO/IEC 14882:2017).
- **Compiler:** MinGW-w64 GCC 16.1.0 (MSYS2 Project).
- **Compilation Flags:**

```
g++ -std=c++17 -Wall -Wextra main.cpp student.cpp attendance.cpp
examination.cpp -o SchoolManagementSystem.exe
```

- **Code Cleanliness:** 0 compiler warnings under `-Wall -Wextra`.
- **Target OS:** Windows 10/11 native 64-bit console binary.

Standard Library Utilization

Header	Role in System
`<vector>`	Dynamic in-memory storage arrays
`<string>`	Safe text handling & tokenization
`<fstream>`	Persistent disk I/O streams
`<sstream>`	Delimited field extraction via `getline`
`<algorithm>`	Sorting (`std::sort`), `remove_if`
`<iomanip>`	Tabular alignment (`setw`, `left`)
`<filesystem>`	Automatic directory tree creation

Win32 Console Integration

Enables modern virtual terminal processing and UTF-8 output on Windows:

```
#ifdef _WIN32
SetConsoleOutputCP(CP_UTF8);
HANDLE hOut = GetStdHandle(
    STD_OUTPUT_HANDLE);
DWORD dwMode = 0;
GetConsoleMode(hOut, &dwMode);
dwMode |= ENABLE_VIRTUAL_
    TERMINAL_PROCESSING;
```

3-Tier Architecture & Execution Lifecycle

Clear separation of concerns from application bootstrap to clean termination

1. Bootstrap & Loading

Step 1: UI::initConsole()
Enables Windows UTF-8 & ANSI terminal modes.

Step 2: Manager Instantiation
Instantiates `studentMgr`, `attendanceMgr`, and `examMgr`.

Step 3: loadFromFile()
Deserializes text files into in-memory vectors. Missing files start empty safely.

2. Interactive Event Loop

Step 4: Main Menu Loop
Displays 3 core module options + Exit. Validates numerical choices.

Step 5: Sub-Menu Routing
- Option 1 → `runStudentMenu()`
- Option 2 → `runAttendanceMenu()`
- Option 3 → `runExaminationMenu()`

Step 6: Defensive Validation
Sanitizes inputs; rejects delimiters and invalid numerical ranges.

3. Real-Time Persistence

Step 7: In-Memory Mutation
Vector items are inserted, updated, or removed using STL algorithms.

Step 8: saveToFile()
Changes are immediately flushed to disk via truncated `std::ofstream`.

Step 9: Clean Termination
Choice 0 terminates the event loop with status code 0 and flushes stdout.

Student Information Management

Full student lifecycle, unique primary key validation, and cascading referential integrity

Core Subsystem Operations

- **Add New Student:** Prompts for ID, Name, Age (4–100), and Class. Strictly prevents duplicate Student IDs via `studentExists(id)`.
- **Display All Students:** Generates an aligned tabular overview of all enrolled students using `std::setw` formatting.
- **Search by ID:** Fast $O(N)$ lookup displaying student records inside an isolated console information box.
- **Update Student Info:** Allows editing of individual fields; pressing Enter keeps existing values intact while keeping ID immutable.
- **Delete Student Record:** Removes the student from memory and disk using the C++ erase-remove idiom.

Cascading Deletion Synchronization

When a student is deleted, the system prevents orphan data across the other two modules:

```
// main.cpp - Cascading Synchronization
if (studentMgr.handleDeleteStudent(deletedId)) {
    // Cascade to Module 2 & Module 3
    attendanceMgr.deleteRecordsForStudent(deletedId);
    examMgr.deleteRecordsForStudent(deletedId);
    UI::printInfo("Synchronized: All associated "
                  "records removed.");
}
```

Why this matters: Guarantees referential integrity across all three database files without needing a foreign key engine.

Attendance Management System

Daily attendance tracking, ISO date enforcement, duplicate protection, and analytics

Key Operations & Verification

- **Cross-Module Student Verification:** Queries `studentMgr.studentExists(id)` before allowing attendance logging.
- **ISO Date Validation:** Enforces `YYYY-MM-DD` format, validating Year (2000–2099), Month (1–12), and Day (1–31).
- **Status Normalization:** Accepts 'P', 'p', 'Present', 'A', 'a', 'Absent' and normalizes them into clean strings.
- **History Inspection:** Supports viewing all school logs or filtering by an individual student's ID.

Same-Day Duplicate Prevention

Prevents marking attendance more than once for the same student on the same date via `hasAttendance(id, date)`.

Percentage Formula & 75% Alert

```
double AttendanceManager::calculateAttendancePercentage(
    const std::string& studentId,
    int& presentCount, int& totalDays) const {
    // Zero-session protection
    if (totalDays == 0) return 0.0;
    return (static_cast<double>(presentCount)
            / totalDays) * 100.0;
}
```

Institutional Attendance Thresholds:

- **≥ 75.0%:** Status = "REGULAR / ELIGIBLE"
- **< 75.0%:** Status = "SHORTAGE / LOW ATTENDANCE" (triggers a warning banner)

Examination Results Management

Subject score recording, range validation, overwrite checks, and automated grading

Marks Validation & Ingestion

- **Student Existence Check:** Verifies Student ID against `StudentManager` before prompting for marks.
- **Marks Boundary Validation:**

```
0.0 ≤ Obtained Marks ≤ Total Marks  (Total Marks ≥ 1.0)
```

- **Subject Overwrite Protection:** If a mark already exists for the subject, the system prompts the user for confirmation before updating.
- **Dynamic Calculations:** Automatically computes percentages and letter grades for each subject upon entry.

Standardized Institutional Grading Scale

Percentage	Grade	Academic Standing
90.0% – 100.0%	**A+**	High Distinction / Outstanding
80.0% – 89.9%	**A**	Distinction / Excellent
70.0% – 79.9%	**B**	Commendable / Good
60.0% – 69.9%	**C**	Satisfactory / Average
50.0% – 59.9%	**D**	Passing / Minimum Competence
Below 50.0%	**F**	Failing / Unsatisfactory

Implemented via `ExaminationManager::calculateGrade(percentage)` using a clean descending conditional decision structure.

Student Profile Card (Console Dashboard)

Cross-subsystem data aggregation formatted in an aligned box-border card

Cross-Subsystem Data Integration

The Profile Card queries all three managers to build a unified profile:

- **[1] Personal Details:** Student ID, Full Name, Age, and Class from `StudentManager`.
- **[2] Attendance Analytics:** Total sessions, Present days, Absent days, Percentage, and Eligibility from `AttendanceManager`.
- **[3] Academic Summary:** Number of subjects, Total marks, Cumulative percentage, Overall grade, and Standing from `ExaminationManager`.

Pixel-Perfect Padding Strategy

Uses custom row printer `printCardRow()` that calculates padding from visible text length, preventing ANSI escape sequence byte misalignments.

```
╔═══════════════════════════════════════════════════════════════╗
║                    OFFICIAL STUDENT PROFILE CARD              ║
║                     SCHOOL MANAGEMENT SYSTEM                  ║
╚═══════════════════════════════════════════════════════════════╝
╔═══════════════════════════════════════════════════════════════╗
║ [1] PERSONAL DETAILS                                          ║
║   Student ID : S101               Class : CS-101              ║
║   Full Name  : Alice Johnson      Age   : 19 years            ║
╚═══════════════════════════════════════════════════════════════╝
╔═══════════════════════════════════════════════════════════════╗
║ [2] ATTENDANCE SUMMARY                                        ║
║   Total Days : 2                  Present Days : 2            ║
║   Absent Days: 0                  Attendance % : 100.0%       ║
║   Status     : REGULAR / ELIGIBLE (>= 75%)                    ║
╚═══════════════════════════════════════════════════════════════╝
╔═══════════════════════════════════════════════════════════════╗
║ [3] EXAMINATION SUMMARY                                       ║
║   Subjects   : 2                  Total Marks  : 186 / 200    ║
║   Percentage : 93.00%             Overall Grade: A+           ║
║   Standing   : OUTSTANDING / PASSED WITH DISTINCTION          ║
╚═══════════════════════════════════════════════════════════════╝
```

Student Academic Report Card

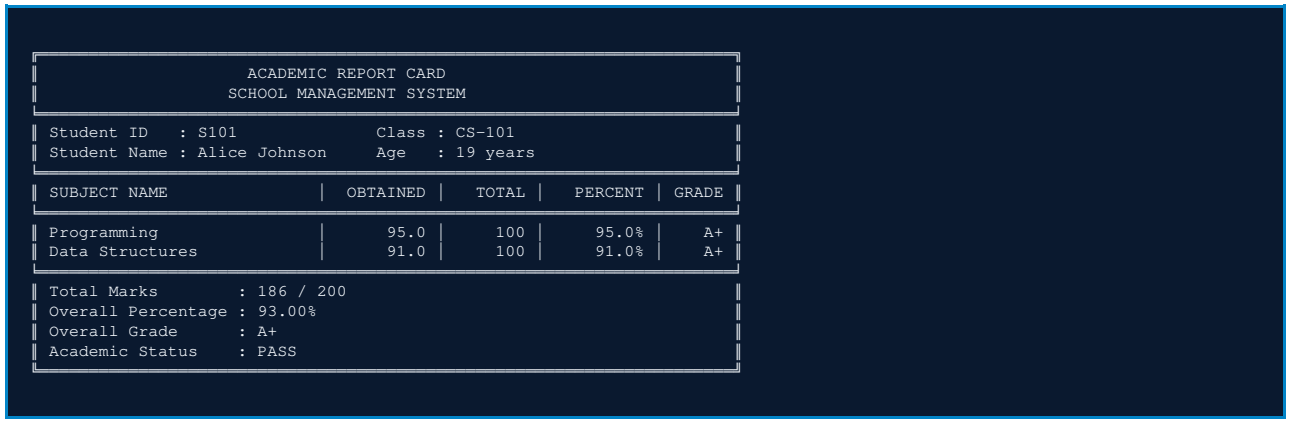
Official transcript formatting, subject breakdowns, aggregate percentage, and PASS/FAIL rules

Transcript Logic & Grading

- **Full Course Breakdown:** Tabulates every subject examined for the student with Obtained Marks, Total Marks, and Individual Grades.
- **Cumulative Metrics:** Sums total obtained marks and total maximum marks to compute the aggregate percentage.
- **Strict PASS / FAIL Criteria:**

```
PASS = (Overall % ≥ 50.0%) && (!hasFailedSubject)
```

  If even a single subject has Grade 'F' (< 50%), the overall verdict is flagged as FAIL.
- **Academic Standing:** Visual status highlights pass/fail states clearly.



```
╔═══════════════════════════════════════════════════════════════╗
║                    ACADEMIC REPORT CARD                       ║
║                  SCHOOL MANAGEMENT SYSTEM                     ║
╚═══════════════════════════════════════════════════════════════╝
 Student ID   : S101              Class : CS-101
 Student Name : Alice Johnson     Age   : 19 years
┌──────────────────────┬──────────┬──────────┬──────────┬───────┐
│ SUBJECT NAME         │ OBTAINED │   TOTAL  │  PERCENT │ GRADE │
├──────────────────────┼──────────┼──────────┼──────────┼───────┤
│ Programming          │     95.0 │      100 │    95.0% │    A+ │
│ Data Structures      │     91.0 │      100 │    91.0% │    A+ │
└──────────────────────┴──────────┴──────────┴──────────┴───────┘
 Total Marks        : 186 / 200
 Overall Percentage : 93.00%
 Overall Grade      : A+
 Academic Status    : PASS
```

Class Merit List & Ranking Algorithm

Sorting by percentage with dense competition ranking for identical scores

Competition Ranking Logic

```
// examination.cpp - Merit List Algorithm
int currentRank = 1;
for (size_t i = 0; i < summaries.size(); ++i) {
    if (i > 0) {
        // Epsilon check for floating-point equality
        if (std::abs(summaries[i].percentage -
                     summaries[i-1].percentage) > 0.0001) {
            currentRank = static_cast<int>(i + 1);
        }
    }
    summaries[i].rank = currentRank;
}
```

How Ties Are Handled:

If Student A has 93% (Rank #1), while Student B and Student C both have 85%, both receive Rank #2. The subsequent student with 80% receives Rank #4.

Live Console Merit Output

```
+------+------------+---------------------------+------------+------------+-------+----+
| Rank | Student ID | Student Name              | Class      | Percentage | Grade | St |
+------+------------+---------------------------+------------+------------+-------+----+
| #1   | S101       | Alice Johnson             | CS-101     |     93.00% |    A+ |  P |
| #2   | S102       | Bob Smith                 | CS-101     |     85.00% |     A |  P |
| #2   | S103       | Charlie Brown             | CS-101     |     85.00% |     A |  P |
+------+------------+---------------------------+------------+------------+-------+----+
Note: Students with equal percentage share the same merit position rank.
Status Indicator: P = All Passed, F = Needs Improvement / Failed Subject
```

Flexible Grouping: The user can select individual cohorts (e.g., "CS-101") or select Option 0 for an institution-wide combined leaderboard.

Object-Oriented Programming (OOP) Implementation

Encapsulation, constructors, Single Responsibility Principle, and class relationships

1. Encapsulation

All core attributes are strictly `private`:

```
class Student {
private:
    std::string id;
    std::string name;
    int age;
    std::string studentClass;
public:
    std::string getId() const;
    void setName(const std::string&);
};
```

Data integrity is protected by restricting state changes exclusively through validated setters.

2. Constructors

Ensures zero-overhead in-place member initialization:

```
// Member Initializer List
Student::Student(
    const std::string& id,
    const std::string& name,
    int age,
    const std::string& c)
    : id(id), name(name),
      age(age), studentClass(c) {}
```

Default constructors provide safe zero-state instantiation for stream deserialization.

3. Composition & SRP

Single Responsibility Principle (SRP):

- `StudentManager`: Manages students.
- `AttendanceManager`: Manages dates & presence.
- `ExaminationManager`: Manages scores & ranks.

File Handling & Data Storage Architecture

Delimited plain text files, automatic directory creation, and synchronized persistence

Flat File Schema Design

Records are stored in plain text files using the pipe delimiter (`'|'`):

File Path	Schema Format	Live Example
`data/students.txt`	`ID|Name|Age|Class`	`S101|Alice Johnson|19|CS-101`
`data/attendance.txt`	`ID|Date|Status`	`S101|2026-11-01|Present`
`data/results.txt`	`ID|Subj|Obt|Total`	`S101|Programming|95|100`

Why Pipe (|) Delimiters?

Names, subject titles, and notes frequently contain spaces and commas. The pipe delimiter avoids string parsing collisions without requiring complex quotation escaping.

Safe Ingestion & Serialization

```
// Automatic Folder Creation (C++17)
std::filesystem::path p(filePath);
if (p.has_parent_path()) {
    std::filesystem::create_directories(p.parent_path());
}

// Truncated Atomic Rewrite
std::ofstream outFile(filePath, std::ios::trunc);
for (const auto& s : students) {
    outFile << s.toFileString() << "\n";
}
outFile.close();
```

- **Crash-Proof First Launch:** If files do not exist, `loadFromFile()` returns cleanly instead of throwing an unhandled exception.
- **Complete Persistence:** All mutations are flushed immediately to disk.

Defensive Programming & Input Validation

Eliminating console stream crashes and protecting file integrity

The Classic `cin >>` Crash Problem

In standard C++ programs, writing:

```
int choice;
std::cin >> choice; // DANGEROUS STREAM FAILURE!
```

If a user enters non-numeric text (e.g., `"abc"`), `cin` sets an internal failbit, locks the stream, and sends terminal menus into an uncontrollable infinite loop.

Delimiter Injection Shielding

Users attempting to enter `|` in names or subject fields are rejected with an explicit warning, ensuring the flat file database format cannot be corrupted.

The System's Sanitization Solution

```
// student.h - Crash-Proof Integer Reading
int getIntInput(const std::string& prompt, int min, int max) {
    while (true) {
        std::string line;
        std::getline(std::cin, line);
        line = trim(line);
        try {
            size_t idx = 0;
            int val = std::stoi(line, &idx);
            // Check for trailing junk (e.g. "12abc")
            if (idx != line.length()) continue;
            if (val >= min && val ≤ max) return val;
        } catch (...) { /* Catch parsing exceptions */ }
        printError("Invalid numeric input.");
    }
}
```

Guarantees that no user input can crash or deadlock the application.

User Experience • Visual Polish

Console UI/UX Design System

ANSI color schemes, UTF-8 box styling, and dynamic tabular alignment

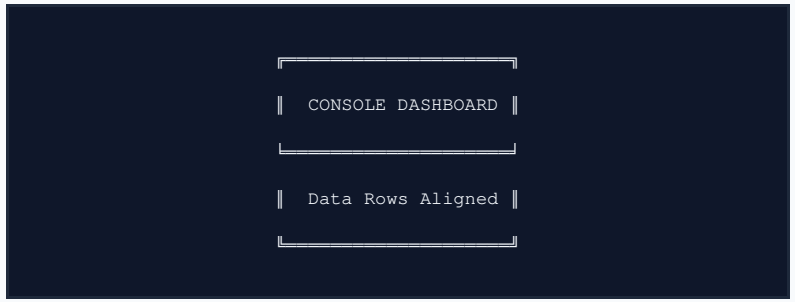
ANSI Color Palette

High-contrast virtual terminal color highlighting:

- **CYAN: Main Headers & Borders**
- **GREEN: Success & Passes**
- **YELLOW: Prompts & Warnings**
- **RED: Errors & Failing Marks**
- **GRAY: Footnotes & Metadata**

UTF-8 Double-Line Borders

Clean box drawing characters configured via Code Page 65001:



Gives the console application an organized, terminal-GUI feel rather than raw unstructured output.

Tabular Stream Formatting

Uses `<iomanip>` stream manipulators for aligned columns:

```
std::cout << "| "
          << std::left << std::setw(10) << id
          << " | "
          << std::setw(25) << name
          << " | "
          << std::right << std::setw(8) << marks
```

System Testing & Verification Matrix

Comprehensive validation of edge cases, data consistency, and crash prevention

Test Scenario	Input Condition Tested	Expected Behavior	S
Duplicate Student ID	Adding existing ID "S101"	Blocks insertion; displays error message	
Age Out-of-Bounds	Entering age < 4 or > 100	Rejects input; reprompts within bounds	
Same-Day Attendance Duplicate	Logging ID "S101" on date "2026-11-01" twice	Rejects second log; shows duplicate alert	
Exam Marks > Total Marks	Obtained = 105, Total = 100	Blocks input; enforces 0 ≤ Obtained ≤ Total	
Subject Score Overwrite	Re-entering marks for "Programming"	Warns user; prompts confirmation before update	
Cascading Referential Integrity	Deleting student "S101"	Purges attendance and exam records from all files	
Stream Crash Protection	Typing "xyz" at integer menu prompt	Catches exception; displays error; no terminal lock	
Reboot Persistence	Terminating and restarting application	Restores in-memory records from disk files	

System Limitations & Future Engineering Roadmap

Clear identification of architectural constraints and planned optimizations

Current Architectural Limitations

- **In-Memory Working Set:** All records are loaded into RAM on startup. Fine for thousands of students; unsuitable for millions.
- **Linear Search Time ($O(N)$):** Lookups iterate sequentially through vectors.
- **Truncated File Rewrites:** Updates rewrite entire files via `ios::trunc` rather than modifying bytes in-place.
- **Single-User Console Process:** No multi-threading, concurrency locks, or multi-user network protocol.

Future Enterprise Roadmap

- **Hash Indexing:** Transition to `std::unordered_map<std::string, Student>` for $O(1)$ constant-time primary key lookups.
- **ACID Storage Engine:** Transition from plain text files to an embedded database like SQLite or a custom B+ Tree on disk.
- **Pagination Support:** Implement cursor-based pagination for console tables with 50+ rows.
- **Multi-Tier Client-Server:** Wrap core business managers into REST / gRPC API services backed by a modern web dashboard.

Defense Summary • Project Conclusion

Core Technical Accomplishments

- 100% compliant with university functional and technology requirements.
- Clean 3-tier object-oriented architecture in standard C++17.
- Defensive input sanitization eliminates console stream crashes.
- Delimited file handling with cascading referential integrity.
- 3 special university features: Profile Card, Report Card, Merit List.

Viva Examination Readiness

Prepared to address all oral questions covering:

- OOP principles (Encapsulation, SRP, constructor initializer lists).
- Stream handling, custom delimiters, and file truncation.
- IntroSort, lambda predicates, and competition ranking logic.
- Architecture design choices and memory scaling trade-offs.

Ready for University Faculty Defense • Live Code Demonstration